Mauricio Salinas

Back-end Developer · Python · FastAPI · Microservicios Event-Driven · Clean Architecture

Santiago, Chile · maundrex@gmail.com · +56 9 4758 6414
mauridev94.github.io · github.com/MauriDev94 · linkedin.com/in/mauricio-salinas-rubio · **Inglés C2 (EF SET 71/100)** · Disponible para trabajo remoto

PERFIL

Desarrollador backend enfocado en **Python**, **APIs REST** y **microservicios event-driven**. Construyo servicios con **FastAPI** y **Django/DRF** aplicando **SOLID, Clean Architecture, DDD y TDD**, con cobertura real: **274 tests al 98.7% branch coverage** en mi proyecto principal y **138 tests** en el sistema de microservicios, con integración vía **testcontainers**. Experiencia en **PostgreSQL**, **RabbitMQ** (idempotencia, dead-letter queues, eventual consistency), autenticación avanzada (JWT/OAuth2/OTP/SSO), **observabilidad** (logging JSON + correlation IDs) y **CI/CD con GitHub Actions**. Inglés C2 para documentación técnica y equipos remotos.

EXPERIENCIA

Desarrollo Backend — Proyectos propios

Santiago, Chile · Abr 2025 – presente

- Diseño y desarrollo de **APIs REST production-grade** con FastAPI y Django/DRF aplicando Clean Architecture, SOLID, DDD y TDD.
- Diseño de **sistemas distribuidos event-driven** con RabbitMQ: idempotencia, dead-letter queues, retries con backoff y consistencia eventual entre servicios.
- Pipelines de **CI/CD** con GitHub Actions (lint, type-checking, tests contra PostgreSQL real, cobertura bloqueante) y despliegue continuo en Render.

Analista de Soporte

Cadaques (Domino) · Santiago, Chile · Abr 2023 – Jul 2024

- **Análisis y resolución de incidencias** en sistemas productivos, con diagnóstico de causa raíz.
- Consultas y mantenimiento de bases de datos **SQL** en producción; reportes estratégicos para el área de operaciones.
- Desarrollo de **scripts Python** para automatización de procesos operativos, reduciendo trabajo manual repetitivo.
- Coordinación con el equipo de desarrollo para el escalamiento y seguimiento de defectos hasta su resolución.
- Monitoreo y configuración de infraestructura de red empresarial.

PROYECTOS

Event-Driven Orders — Microservicios con RabbitMQ

> **`3 microservicios · 138 tests · cobertura 85–92% por servicio`**

`Python 3.12 · FastAPI · RabbitMQ (aio-pika) · PostgreSQL 16 · Pydantic v2 · Docker Compose · pytest · testcontainers · structlog · GitHub Actions`

Procesamiento de órdenes e-commerce con **3 microservicios** (order, inventory, notification) que se comunican solo por **eventos** vía RabbitMQ, con Clean Architecture + DDD en cada servicio.

- **Idempotencia** en los consumers (`processed_events` con `INSERT ... ON CONFLICT DO NOTHING`): un evento redelivered no duplica efectos de negocio.
- **Atomicidad anti race-condition** en la reserva de stock (`UPDATE` condicional + row-level locking + `SAVEPOINT`).
- **Dead-letter queues + retries con backoff** (3 etapas) con clasificación de fallos transitorios/permanentes.
- **Eventual consistency** por coreografía de eventos, sin transacción distribuida.
- **Observabilidad**: logging estructurado JSON + **correlation ID end-to-end** (HTTP → eventos → 3 servicios) y reconexión robusta al broker.
- **database-per-service**; **138 tests**, cobertura **85–92%** por servicio — integración contra **PostgreSQL real vía testcontainers** (SQLite carece del row-level locking necesario para verificar concurrencia).
- **CI/CD**: GitHub Actions con matrix por servicio (lint + mypy + tests).

Repo: github.com/MauriDev94/event-driven-orders

MedBook API — Reservas médicas con Django + DRF

> **`274 tests · 98.7% branch coverage · 0 fallos · TDD estricto`**

`Python 3.12 · Django 5.1 · DRF · PostgreSQL 16 · simplejwt · drf-spectacular · pytest · factory-boy · Docker · GitHub Actions · Codecov`

API REST de citas médicas que cubre el ciclo completo: los médicos publican horarios, el sistema genera slots y los pacientes reservan, avanzando por una **máquina de estados** (pending → confirmed → completed / cancelled / no-show).

- **274 tests · 98.7% branch coverage · 0 fallos** — desarrollado con **TDD estricto** (Red-Green-Refactor) y `--cov-fail-under` enforced en CI.
- **RBAC** con 8 clases de permisos y control de acceso por rol (médico/paciente/admin) a nivel de objeto.
- **Transacciones atómicas + UPDATE atómico** para prevenir *race conditions* al reservar slots concurrentes.
- Optimización ORM: `select_related`/`prefetch_related` (cero N+1), `annotate()`/`Q()` para queries.
- Documentación **OpenAPI 3** con drf-spectacular (Swagger UI) y notificaciones por email (django-anymail/Resend).
- **Rate limiting** (AnonRateThrottle) para protección anti fuerza-bruta en login.
- **CI/CD**: GitHub Actions con lint (ruff), tests contra PostgreSQL real y cobertura a Codecov.

Repo: github.com/MauriDev94/MedBook · Live (Swagger UI): medbook-oiio.onrender.com/api/docs

API Monolith — Monolito modular con Clean Architecture

> **`3 módulos de dominio · Auth JWT/OAuth2/OTP/SSO · branch coverage con gate en 82%`**

`Python · FastAPI · PostgreSQL · SQLAlchemy · Alembic · Pytest · Docker · JWT · OAuth2 · OTP · SSO · GitHub Actions · Render`

API REST con **FastAPI**, Clean Architecture, SOLID y DDD. **3 módulos de dominio** (Auth, Users, Todos), cada uno con capas `domain / application / infrastructure / presentation / di`. Auth incluye JWT/OAuth2, OTP, reset de contraseña y Google login vía SSO.

- Cada módulo define sus **propios contratos** (`application/contracts/`) con regla de dependencia estricta: **el dominio no tiene dependencias de framework** y la infraestructura es intercambiable sin tocar la lógica de negocio.
- Middleware transversal (logging, errores, seguridad); esquemas PostgreSQL con **SQLAlchemy** y migraciones con **Alembic**.
- **CI/CD** en dos jobs: *quality* (ruff, black, mypy) y *tests* contra PostgreSQL real con `--cov-branch --cov-fail-under=82`. **Desplegada en producción en Render.**

Repo: github.com/MauriDev94/Api_monolith · Live (Swagger UI): api-monolith.onrender.com/docs

HABILIDADES TÉCNICAS

- **Lenguajes:** Python, SQL
- **Frameworks:** FastAPI, Django, Django REST Framework (DRF)
- **Mensajería / event-driven:** RabbitMQ (aio-pika), idempotencia, dead-letter queues, retries con backoff, eventual consistency
- **Bases de datos:** PostgreSQL, MongoDB — diseño de esquemas y consultas, optimización ORM (cero N+1)
- **ORM y migraciones:** SQLAlchemy, Alembic, Django ORM
- **Testing:** Pytest, pytest-django, TDD, factory-boy, testcontainers, pruebas unitarias e integración, coverage.py, quality gates en CI
- **Auth y seguridad:** JWT, OAuth2, OTP, SSO, Google Login, RBAC, rate limiting (throttling)
- **API:** REST, OpenAPI 3 / Swagger (drf-spectacular), diseño de endpoints, versionado
- **CI/CD:** GitHub Actions (lint, tests, cobertura, artefactos), Codecov, pre-commit
- **DevOps:** Docker, Git, GitHub, Render (despliegue continuo)
- **Observabilidad:** structlog (logging JSON), correlation IDs end-to-end
- **Calidad y metodologías:** SOLID, patrones de diseño, Clean Code, ruff, black · Clean Architecture, DDD, TDD, SDD, Microservicios event-driven, database-per-service, Modular Monolith

IDIOMAS

- **Español:** Nativo
- **Inglés: C2 Proficient — EF SET 71/100** (documentación técnica y comunicación con equipos remotos)

EDUCACIÓN

Analista Programador — Instituto Profesional Santo Tomás · Santiago · Mar 2021 – Jul 2025